

UrbanPulse: Kafka Ingestion, Reliability & Priority Consumer Design

Topic Contracts · Partition Sizing · Delivery Semantics · Consumer Group Isolation · Stream Enrichment & Dead-Letter Handling

COURSE CODE	TARGET CITY	DELIVERABLE	LEAD ARCHITECT
DSE ZG556 / CC ZG556	MetroConnect (Pop. 4.2M)	Task B — Ingestion Layer	Streaming Solutions Architect

1 What This Layer Does

The ingestion layer is the single entry point into UrbanPulse. Every downstream guarantee the platform makes — the 90-second traffic signal adaptation SLA, the sub-2-minute AQI advisory, the 15-minute ward energy rollup, and the annual state energy audit — depends on this layer accepting roughly **3,940 events per second** from four heterogeneous sensor fleets, ordering them correctly, retaining them for exactly as long as the regulation demands, and refusing to let one bad payload or one slow analytics consumer stall an emergency pipeline.

Apache Kafka was selected as the ingestion substrate because it decouples producers from consumers in time and in speed. Sensors publish at their own cadence; Flink, Spark, and the dashboard consumers each read the same immutable log at their own pace, from their own committed offset. That property is what makes the priority-consumer design in Section 4 possible at all.

STAGE 1	STAGE 2	STAGE 3	STAGE 4	STAGE 5
Source Fleets 12,000 bus GPS units — 2,400 evt/s 3,800 signal controllers — 380 evt/s 600 AQI sensors — 60 evt/s 1.1M smart meters — 1,100 evt/s	Producers & Keying - JSON payload serialisation - Partition key selection - acks / retry / idempotence config - Schema & range pre-checks	Kafka Cluster 4 domain topics, RF = 3 40 partitions total - Per-topic retention policy - min.insync.replicas = 2	Consumer Tiers - HIGH_PRIORITY control group - STANDARD_PRIORITY analytics - Independent offset tracking - Per-group lag alerting	Enrichment & DLQ - Kafka Streams KTable join - Validation & error tagging - Routing to urbanpulse.dlq - Clean stream to Flink / Spark

2 Data Contract & Topic Configuration

Each sensor domain gets its own topic rather than a shared bus. This isolates retention policy, partition count, and failure blast radius per domain — a retention change demanded by the state energy audit should never touch bus telemetry, and a poison payload in the AQI feed should never sit in the same log as signal control commands.

TOPIC	PARTS.	RF	RETENTION	PARTITION KEY	DESIGN JUSTIFICATION
urbanpulse. bus_gps	12	3	24 hours	route_id	Highest-volume telemetry feed at 2,400 evt/s, giving ~200 evt/s per partition and headroom for Flink parallelism. Keying on route_id guarantees strict per-route ordering, which the bus-bunching detector depends on to compare consecutive positions of two vehicles on the same route. Retention is deliberately short: GPS pings have no regulatory value beyond same-day incident replay, and 24h keeps broker disk bounded against the highest event rate on the cluster.
urbanpulse. signals	8	3	7 days	junction_id	Modest volume (380 evt/s) but the most latency-critical topic. Keying on junction_id keeps each junction's state machine on one partition so phase changes are never reordered. Seven days covers a full weekly traffic cycle, which is the operational window engineers use when retuning signal timing plans.
urbanpulse. air_quality	4	3	90 days	sensor_id	Lowest throughput (60 evt/s) so a small partition count avoids needless metadata overhead. A 90-day window spans a complete seasonal pollution cycle, supporting trend analysis and health advisory reporting; anything older is served from the historical Parquet store rather than Kafka.

TOPIC	PARTS.	RF	RETENTION	PARTITION KEY	DESIGN JUSTIFICATION
urbanpulse.smart_meters	16	3	365 days	ward_id	Highest partition count on the cluster — not because throughput is highest, but because ward aggregation is embarrassingly parallel and Spark consumer parallelism is capped by partition count. Keying on ward_id co-locates all meters in a ward on one partition, so the 15-minute tumbling window aggregates without a cross-partition shuffle. The 365-day retention is a regulatory requirement for annual energy audits and billing reconciliation.

Sizing note — why 365-day Kafka retention shaped the wider architecture

At 1,100 events/sec, the smart-meter topic accumulates roughly **35 billion events per year**. Assuming a conservative 200-byte JSON envelope, that is approximately **7 TB of raw log, or ~21 TB once replicated at RF = 3**. This figure is the concrete reason the platform offloads historical analytics to columnar Parquet object storage instead of relying on Kafka replay for long-range reprocessing — the same arithmetic that drives the Lambda-style split documented in the Task A architecture.

3 Producer Design & Delivery Semantics

Producers are not thin wrappers around a socket. Each one encodes a deliberate position on the durability-versus-latency trade-off, and each one is responsible for the partition key that determines downstream ordering.

PRODUCER	KEY STRATEGY	DELIVERY CONFIGURATION	WHAT IT GUARANTEES, AND WHAT IT DELIBERATELY DOES NOT
Bus GPS Producer	route_id	acks=all, idempotent producer enabled	Guarantees strict ordering of position reports within a route, which is a hard prerequisite for the Flink bunching logic: detecting that two buses stayed within 200 m for five minutes is only sound if their position history arrives in order. It does not guarantee global ordering across routes, and does not need to — no rule in the system compares buses on different routes.
Air Quality Producer	sensor_id	acks='all', retries=5 — at-least-once semantics	Chooses durability over de-duplication: a retrieved AQI reading may be delivered twice, but an emergency advisory must never be silently dropped because a broker was mid-election. The producer also simulates a 5% null-AQI fault rate to exercise the validation path. Critically, the producer logs these faults and forwards them rather than discarding them — the decision to quarantine belongs to the validation stage, not the edge.
Signal Producer	junction_id	acks=all, min.insync.replicas=2	Preserves the ordering of phase transitions per junction. Writing to at least two in-sync replicas before acknowledging means a single broker loss cannot lose a control event.
Smart Meter Producer	ward_id	acks=all, batched / compressed	Optimised for throughput rather than latency. Batching 1.1 M meters into ward-keyed partitions trades a few hundred milliseconds of send latency for a large reduction in request overhead — an acceptable trade because nothing downstream of this topic has a sub-second SLA.

Why faults are forwarded rather than dropped at the producer

A producer that silently swallows a malformed reading destroys evidence. The 5% null-AQI events are passed downstream precisely so that the validation stage can attach an error_reason and route them to the dead-letter topic, where a sensor-maintenance crew can see which sensor is failing and how often. The DLQ therefore doubles as a hardware fault-detection signal, not merely an error bin.

4 Priority Consumer Architecture

The urbanpulse.signals topic serves two consumers with incompatible service levels: a control system bound by a 90-second adaptation SLA, and an analytics dashboard that can tolerate minutes of lag. The design problem is to ensure the second can never degrade the first.

The solution exploits a structural property of Kafka: **consumer groups track offsets independently**. Two groups reading the same topic maintain entirely separate committed positions and separate lag. A slow group accumulates its own lag; it applies no backpressure to the broker, consumes no capacity from the other group's fetch path, and cannot delay another group's commits.

HIGH PRIORITY Signal Control Group

Composition: a single dedicated consumer subscribed to all 8 partitions of `urbanpulse.signals`.

Consumes: real-time phase and occupancy events feeding the SCADA signal-control interface.

Behaviour under load: maintains near-zero lag. Because it is the sole member of its group, no rebalance is triggered by activity elsewhere, and its offset advances independently of any analytics workload.

Design rationale: a single consumer avoids rebalance pauses entirely. At 380 evt/s the topic is well within one consumer's capacity, so partition-level parallelism would buy throughput the SLA does not need while introducing rebalance risk it cannot tolerate.

STANDARD Analytics Dashboard Group

Composition: three consumers sharing the 8 partitions of the same topic.

Consumes: the same event stream, feeding aggregate dashboards for ward officers.

Behaviour under load: during a simulated processing slowdown this group's lag grows measurably — and the high-priority group's lag does not move. That divergence is the evidence the isolation works.

Design rationale: three consumers give the analytics tier horizontal headroom to catch up after a slowdown, since it is the tier that will actually fall behind. Recovery here is a throughput problem, not a latency problem.

Verification performed

Artificial processing delay was injected into the STANDARD_PRIORITY consumers while both groups read the live topic. Consumer-group lag was sampled for both groups throughout. The standard group's lag rose as expected while the high-priority group's lag remained flat at near zero, confirming that offset isolation — not merely resource over-provisioning — is what protects the 90-second SLA.

5 Stream Enrichment and Dead-Letter Isolation

5.1 Enrichment: joining telemetry to reference data

Raw GPS pings carry position but no schedule context, so no consumer can compute whether a bus is early or late without a second lookup. Rather than forcing every downstream consumer to perform that lookup, a Kafka Streams job enriches the stream once, at ingestion.

The job joins the `bus_gps` `KStream` against a static `route_schedule` `KTable`, appending `scheduled_arrival_time` and `terminal` to each payload. The stream/table join is the correct primitive here because the two inputs have fundamentally different semantics: GPS pings are an unbounded sequence of facts, while the schedule is a slowly-changing lookup whose latest value per key is all that matters. The `KTable` holds that latest-value view locally, so the join adds no network round-trip per event and no external database becomes a bottleneck in the hot path.

5.2 Dead-letter queue: containing poison pills

Roughly 5% of AQI readings arrive with a null value. Passed through untouched, a null would either crash a Flink operator or, worse, silently corrupt an averaged pollution figure that later appears in a government submission. Validation therefore sits between ingestion and processing.

STEP	MECHANISM	EFFECT ON THE PLATFORM
1. Validate	Schema, null and range checks applied per record	Malformed, null-valued and out-of-range sensor payloads are identified before they reach any stateful operator.
2. Tag	An <code>error_reason</code> field is injected into the failing record	The quarantined message carries its own diagnosis, so triage does not require re-running the validator or correlating logs by timestamp.
3. Route	Failing records are branched to <code>urbanpulse.dlq</code>	The main stream continues uninterrupted. A single poison pill cannot crash the Flink processing layer or stall a partition behind an un-committable offset.
4. Monitor	DLQ volume and per-sensor error counts are alerted on	A sustained rise in DLQ traffic from one <code>sensor_id</code> becomes a maintenance ticket — the DLQ operates as a field-hardware health signal.

6 Failure Modes Addressed and How Each Was Verified

FAILURE MODE	MITIGATION BUILT INTO THIS LAYER	VERIFICATION METHOD
Single broker hardware failure	Replication factor 3 with <code>min.insync.replicas=2</code> across all four topics	Broker shutdown test — confirm zero data loss and continued producer acknowledgement
Malformed / null sensor payload	Validation stage with <code>error_reason</code> tagging and routing to <code>urbanpulse.dlq</code>	Poison-pill injection at a 5% rate; confirm main stream continues and DLQ receives the failures

FAILURE MODE	MITIGATION BUILT INTO THIS LAYER	VERIFICATION METHOD
Analytics workload starving control workload	Separate consumer groups with independent offset tracking	Induced slowdown on the standard group; compare lag curves of both groups
Out-of-order events breaking detection logic	Deterministic partition keys (route_id , junction_id , sensor_id , ward_id)	Per-key offset sequence inspection to confirm ordering within each partition
Duplicate alerts after a network partition	Idempotent producers combined with downstream checkpointing	Forced producer retry; confirm no duplicate records committed to the partition
Unbounded broker disk growth	Per-topic retention (24 h / 7 d / 90 d / 365 d) enforced at topic configuration level	Automated retention policy audit against the documented data contract

Summary

The ingestion layer converts four unrelated sensor fleets into four governed, ordered, independently retained event logs. Partition counts are sized to downstream parallelism rather than raw throughput; keys are chosen to make ordering guarantees match the analytical questions being asked; consumer groups are separated so that service levels are structurally isolated rather than merely resourced; and invalid data is quarantined with its diagnosis attached instead of being dropped or allowed to propagate. Every guarantee the processing layer offers rests on these four properties.